Learning Inverse Kinematics by Imitating a Numerical Solver: Analysis and Improvements on Two 3-DOF Manipulators

Durgesh S. Reddiyar
Ira A. Fulton Schools of Engineering
Arizona State University, Tempe, AZ, USA
dreddiya@asu.edu

***Abstract*—To move a robot arm's hand to a target, we need its inverse kinematics (IK): the joint values that place the hand there. A popular shortcut is to train a neural network for IK, using an existing numerical IK solver to create the training examples. We build such a system with the MATLAB Robotics Toolbox and Keras for two simple three-joint arms, and measure how far the arm's hand actually lands from each target. The baseline network misses by a median of about 4% of the arm's reach, and by more than half the reach for the worst 5% of targets. Keras's default "accuracy" metric still gives it 74%, because that metric only checks which joint value is largest. Most of the error comes from the training data. The solver often picks different but equally valid solutions for neighbouring targets, so the network has to learn an inconsistent mapping. With consistent, exactly computed labels and a sine–cosine output, the same network becomes $43\times$ more accurate at the median and about $200\times$ more accurate on the hardest targets. Used as a starting guess, it cuts the solver's iterations by $3.5\times$ and always gives the intended solution.**

***Index Terms*—Inverse kinematics, neural networks, robot manipulators, training data, evaluation metrics.**

I. Introduction

A robot arm is controlled through its joints, but tasks are described by where the hand should go. Going from joint values to hand position is easy: it is a fixed chain of rotations and translations called forward kinematics (FK) [1]. Going the other way, from a target position to joint values, is called inverse kinematics (IK) and is harder. A target can have no solution, several, or infinitely many. Some arms have exact formulas for IK [2], [3]. Most practical tools instead use iterative solvers that repeatedly adjust the joints until the hand reaches the target [4]–[8].

Neural networks offer another option [9]–[11]: once trained, a network gives an answer in one fast, fixed-time step. A common recipe is to pick random targets, label each one by running a numerical solver, and train a network on these examples. The catch is that a target often has several valid solutions, and a network trained on a mix of them can learn their average, which is usually not a solution at all [12].

This report builds and analyses such a pipeline for two textbook three-joint arms [1], using Peter Corke's Robotics Toolbox for MATLAB [13], [14] and Keras [15]. We make three contributions:

- We measure the baseline network by where the hand actually lands, and show that Keras's default accuracy metric hides large errors (Section IV).
- We trace the errors to their causes in the training data: inconsistent solution choices, unwrapped angles, and a sampling bug (Section IV).
- In a controlled study, we show which fixes matter and by how much, and how a trained network can speed up and steer a numerical solver (Section V).

The code that produces every number, table, and figure is released with this report.

II. Related Work

Numerical IK. Most solvers linearise FK at the current guess and take a step towards the target. Plain steps become unstable near singular poses, where the arm loses the ability to move in some direction. Damped least squares and the Levenberg–Marquardt (LM) method add a damping term that keeps steps safe [4], [5], [7], [16], [17]. The toolbox's `ikine` function used here is an LM solver [14]. Two of its properties matter for us. First, which solution it returns depends on the starting guess. Second, it does not wrap angles, so θ and $\theta + 2\pi$ are reported as different answers.

Learned IK. Early work trained networks to map positions straight to joint values [9]. Jordan and Rumelhart showed that this averages multiple solutions into invalid ones [12]. Later methods split the problem into local pieces [10], [11], [18], predict structured outputs [19], or learn the full set of solutions with mixture models [20], invertible networks [21], or normalising flows [22]. Predicting angles directly is also awkward because of the jump at $\pm\pi$; predicting their sine and cosine avoids it [23].

III. Robots and Baseline Pipeline

RRR arm. The first arm has three rotary joints: a base that turns about the vertical axis and two links of length $a_2 = a_3 = 10$ length units (LU). Its hand position is

$$\mathbf{p} = \begin{bmatrix} c_1 r \\ s_1 r \\ a_2 s_2 + a_3 s_{23} \end{bmatrix}, \qquad r = a_2 c_2 + a_3 c_{23}, \tag{1}$$

where $c_i = \cos\theta_i$, $s_i = \sin\theta_i$, and $c_{23} = \cos(\theta_2 + \theta_3)$. The arm can reach any point within 20 LU of its shoulder. Most reachable targets have four solutions: the shoulder can face the target or face away ($\sigma_s = \pm 1$), and the elbow can bend up or down ($\sigma_e = \pm 1$). With $\rho = \sqrt{p_x^2 + p_y^2}$, the exact solutions are

$$\begin{aligned} \theta_1 &= \operatorname{atan2}(p_y, p_x) + \tfrac{1-\sigma_s}{2}\pi, \\ c_3 &= \left(\|\mathbf{p}\|^2 - a_2^2 - a_3^2\right)/(2a_2a_3), \\ \theta_3 &= \operatorname{atan2}\left(\sigma_e\sqrt{1-c_3^2},\, c_3\right), \\ \theta_2 &= \operatorname{atan2}(p_z,\, \sigma_s\rho) - \operatorname{atan2}(a_3s_3,\, a_2 + a_3c_3). \end{aligned} \tag{2}$$

The arm is singular on the vertical axis ($\rho = 0$), on the edge of its reach ($\|\mathbf{p}\| = 20$), and at the origin, where the elbow folds back on itself.

RPP arm. The second arm has a rotary base on a column of height $d_1 = 5$, a vertical slider d_2, and a horizontal slider d_3. Its hand is at $\mathbf{p} = [-s_1d_3,\ c_1d_3,\ d_1 + d_2]^\top$, so its IK has a single solution: $\theta_1 = \operatorname{atan2}(-p_x, p_y)$, $d_2 = p_z - d_1$, and $d_3 = \rho$.

Data. For the RRR arm, 10,000 targets are drawn uniformly from the cube $[-15, 15]^3$. Each is solved with `ikine`, starting from all-zero joints, using a position-only mask and at most 1,000 iterations. Targets with no solution are dropped, leaving 9,181 examples. For the RPP arm, 10,000 targets are drawn with $x \in [0, 2\pi]$ and $y, z \in [5, 200]$.

Network. A fully connected network maps (x, y, z) to three joint values through hidden layers of sizes 32, 64, 128, 256, 256, 256, 128, 128, and 64, with 233,027 parameters in total. All layers use ReLU except one sigmoid layer. Training minimises the mean absolute error (MAE) with Adam [24] (learning rate 10^{-3}, batch size 768) on an 80:20 train/test split. The RRR network is trained for 20,000 epochs.

How we measure error. What matters for a task is where the hand ends up. We therefore pass each predicted joint vector through FK and report the *position error* $\|\mathrm{FK}(\hat{\mathbf{q}}) - \mathbf{p}\|$ in LU. Unlike joint errors, this does not penalise a network for choosing another valid solution or for adding 2π to an angle.

Solver re-implementation. To study the solver at scale, we re-implemented the toolbox's LM method in Python. Each step solves $(J^\top J + \lambda I)\Delta\mathbf{q} = J^\top\mathbf{e}$, where J is the Jacobian and $\mathbf{e}$ is the position error. The damping λ starts at 0.1, is halved after a successful step, and is doubled after a failed one. The method gives up after 1,000 iterations or 100 failed steps in a row.

IV. Analysis of the Baseline

A. Why the solver fails on some targets

The solver found no answer for 8.2% of the RRR targets. The reason is simple: the sampling cube's corners lie up to 26 LU away, so 7.9% of the cube is out of the arm's reach (Figure 1a). Our solver replica failed on 7.8% of a fresh set of targets, and every failure was out of reach. On reachable targets it never failed and needed at most 51 iterations, so the 1,000-iteration limit never mattered. Singular poses did not cause failures either: the kept data come within 0.14 LU of the vertical axis and within 2×10^{-4} LU of the edge of the reach. The solver's answers are accurate, with all position residuals below 5.3×10^{-3} LU.

B. The labels are inconsistent

Because every solve starts from all-zero joints, which of the four solutions the solver returns depends on where the target lies in a complicated pattern. The nearest training target to a test target is typically only 0.84 LU away, yet it uses the same solution type only 66% of the time (Figure 1b). In addition, 1.3% of labels contain an angle outside $(-\pi, \pi]$, up to 71.9 rad (Figure 1c). To the network, these look like completely different answers for nearly identical inputs.

C. The RPP data cover a thin slice of space

The RPP generator draws the x coordinate from $[0, 2\pi]$, an angle range, instead of a position range. As a result, the training targets lie in a thin slab, and the base angle only spans $-49°$ to $0°$, about 14% of a full turn. Inside this slab the RPP network is accurate, with a median error of 2.1 LU on a workspace of about 200 LU. Outside it, the error grows quickly: for the target $[200, 200, 100]$ it is 222 LU.

D. Keras's "accuracy" is not an IK metric

For a network with three outputs and an MAE loss, Keras's default `'accuracy'` only checks whether the *largest* of the three predicted joint values is in the same position as in the label. It says nothing about the size of the error. The RRR network scores 74.1% on its test split and 97.3% on its training split. An RPP network trained on an earlier dataset, with $x \in [0, 360]$, scores 91.4% even though its median position error is 148 LU. Always guessing the most common position of the largest value would already score 81% there.

E. How accurate the baseline really is

Table I lists all trained models. On test targets, the best RRR network (`model20000_rrr`) lands a median of 0.78 LU from the target, or 3.9% of the reach. A network that always outputs the average joint values would miss by 23.2 LU, so the baseline clearly learns a lot. The worst cases, however, are bad. The 95th-percentile error is 10.9 LU, more than half the reach, and only 0.5% of targets are reached within 0.1 LU. The network fits its training targets much better (median 0.50 LU), so it partly memorises them. Most large errors come from picking a different solution type than the label: when the network matches the label's solution type (69% of targets), its median error is 0.60 LU; when it does not, the median is 1.51 LU and the 95th percentile is 19.7 LU. Errors are also largest near the origin, where the folded elbow makes the joint angles poorly defined. On a typical target such as $\mathbf{p} = [1, 4, 9]$, the network predicts $[78.8°, 4.4°, 121.2°]$, against the exact solution $[76.0°, 5.1°, 120.7°]$, and misses by 0.23 LU.

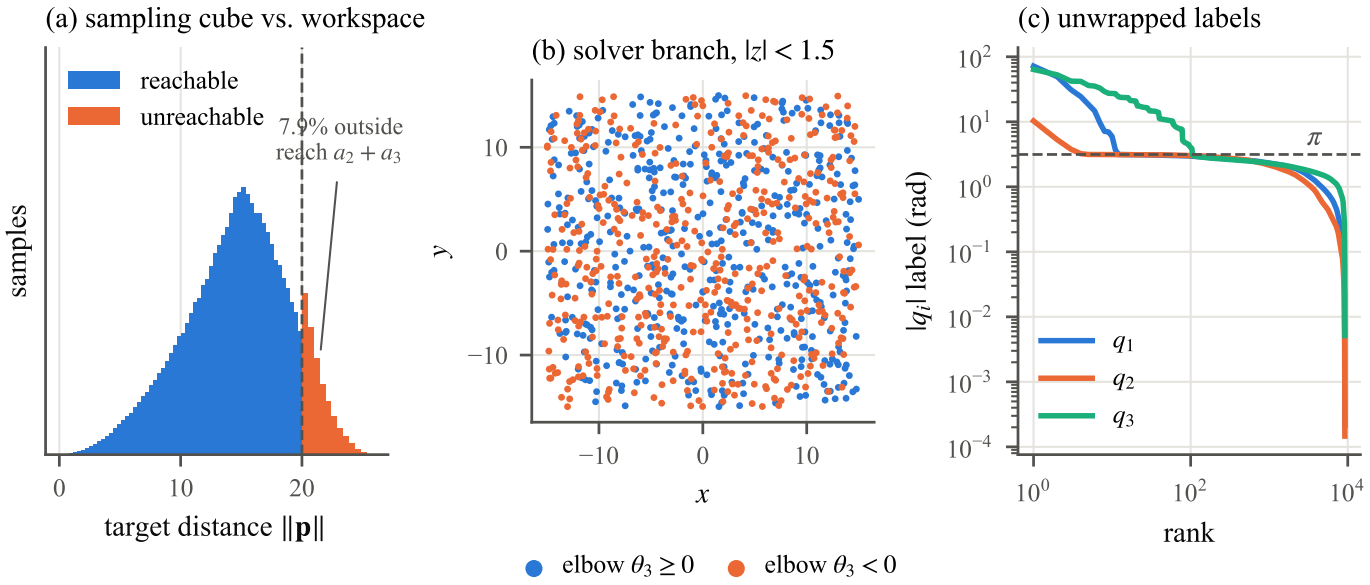


Fig. 1. Problems in the RRR training data. **(a)** Distances of targets drawn from the sampling cube. The 7.9% beyond the arm's 20 LU reach cannot be solved. **(b)** Elbow direction in the solver's answers for targets near $z = 0$. Neighbouring targets often get opposite elbows. **(c)** Sorted joint labels: some exceed π many times over because the solver does not wrap angles.

TABLE I
TRAINED BASELINE MODELS. "KERAS ACC." IS KERAS'S DEFAULT ACCURACY METRIC; "MAJ." IS THE SCORE OF ALWAYS GUESSING THE MOST COMMON ANSWER. JOINT ERRORS ARE WRAPPED TO $(-\pi, \pi]$. POSITION ERRORS ARE IN LU; THE RRR ARM'S REACH IS 20 LU AND THE RPP WORKSPACE IS ABOUT 200 LU. THE LAST COLUMN IS THE SHARE OF TARGETS REACHED WITHIN 1 LU.

Model	Arm	Split	Keras acc. [%]	Maj. [%]	Joint error [deg] (q_1, q_2, q_3)	Position error [LU] median	mean	p95	< 1 LU [%]
`model2000_rrr`	RRR	test	67.7	37.3	53, 46, 74	17.96	18.26	31.89	1.1
`model10000_rrr`	RRR	test	74.0	37.3	27, 34, 33	0.96	2.67	13.98	51.9
`model15000_rrr`	RRR	test	74.4	37.3	26, 34, 33	0.79	2.30	11.72	60.7
`model20000_rrr`	RRR	test	74.1	37.3	27, 33, 32	0.78	2.19	10.92	61.6
`model20000_rrr`	RRR	train	97.3	37.3	4, 4, 5	0.50	0.85	2.14	82.5
`model8000_rpp_rad`	RPP	test	99.9	51.7	1.5 (q_1)	2.11	2.87	7.70	–
`model8000_rpp`	RPP	test	91.4	81.0	27.4 (q_1)	148.16	155.55	352.60	–

V. FIXING THE TRAINING DATA

A. Controlled study

To see which problems matter, we train the same network on the *same* 7,344 training targets and change only the labels, the loss, or the output format. The network has three hidden layers of 256 ReLU units. It is trained with Adam for 40k steps (batch size 512, learning rate 2×10^{-3} with cosine decay) in PyTorch [25], with three random seeds per setting. We test on 20,000 new reachable targets. The five label settings are:

(i) the solver's labels as generated;
(ii) the same labels with angles wrapped to $(-\pi, \pi]$;
(iii) exact labels from Equation (2) with a randomly chosen solution type per target;
(iv) exact labels that always use the same solution type (shoulder facing the target, elbow up);
(v) as (iv), but the network predicts $(\cos\theta_i, \sin\theta_i)$ instead of θ_i.

Table II and Figure 2a show two clear groups. Every setting trained on solver labels or mixed solution types misses by a median of 1.6–16 LU, and its 95th-percentile error is larger than the arm's reach. Every setting trained on one consistent solution type misses by less than 0.075 LU. In more detail:

- *Mixing solution types is the main problem.* With a random solution type per target, the network misses by 14–16 LU, because it learns averages of valid answers that are not answers themselves.
- *The loss only matters when labels are mixed.* With solver labels, the MAE loss is 3× better than the squared loss (1.94 vs. 6.27 LU), because MAE tends to pick one of the competing answers instead of averaging them. With consistent labels, both losses work well.
- *Wrapping angles helps a little.* It improves the median by 15% but leaves the worst cases unchanged.
- *Sine–cosine outputs finish the job.* They remove the jump

TABLE II

CONTROLLED STUDY ON THE RRR ARM. ALL RUNS USE THE SAME 7,344 TRAINING TARGETS AND THE SAME NETWORK; ONLY THE LABELS, LOSS, AND OUTPUT FORMAT CHANGE. POSITION ERRORS IN LU ON 20K NEW REACHABLE TARGETS (MEAN ± S.D. OVER 3 SEEDS). "<0.1" IS THE SHARE OF TARGETS REACHED WITHIN 0.1 LU, IN %.

	MAE (L1) loss			Squared (L2) loss		
Labels / output	median	p95	<0.1	median	p95	<0.1
Solver labels, as generated	1.940 ± 0.074	21.88 ± 0.34	6.8	6.273 ± 0.068	23.98 ± 0.03	0.1
Solver labels, wrapped to $(-\pi, \pi]$	1.646 ± 0.110	22.38 ± 0.19	7.1	5.700 ± 0.041	24.06 ± 0.07	0.1
Closed-form, random branch per sample	14.411 ± 0.051	29.13 ± 0.18	0.0	15.952 ± 0.198	29.83 ± 0.10	0.0
Closed-form, one branch, angle output	0.053 ± 0.001	0.13 ± 0.00	88.2	0.074 ± 0.002	0.17 ± 0.00	72.1
Closed-form, one branch, $(\cos, \sin)$ output	0.045 ± 0.000	0.11 ± 0.00	93.1	0.042 ± 0.001	0.10 ± 0.00	95.0

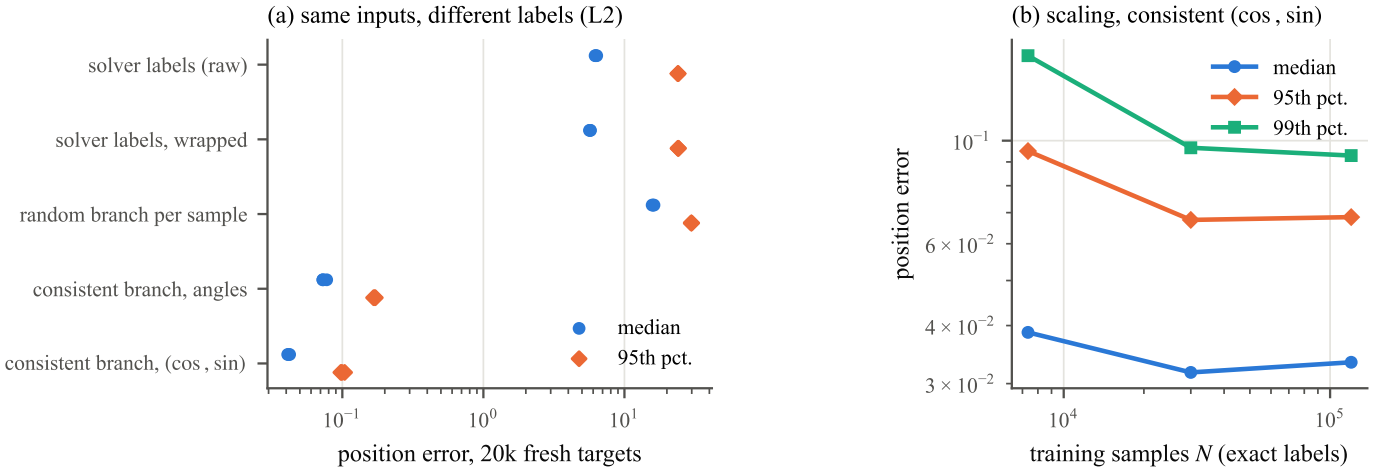


Fig. 2. **(a)** Median and 95th-percentile position error for each label setting (L2 loss, three seeds). **(b)** Error of the improved network as the number of exact training examples grows (mean of three seeds).

at $\pm\pi$ and raise the share of targets reached within 0.1 LU from 72% to 95% (L2 loss).

With the inputs fixed, consistent labels and a sine–cosine output make the network 43× more accurate at the median (1.94 to 0.045 LU) and 199× more accurate at the 95th percentile (21.9 to 0.11 LU).

B. More data

Because exact labels are cheap, we can also train on more targets (Figure 2b). With 30,000 examples and 60k training steps, the improved network misses by a median of 0.032 LU and reaches 99.1% of targets within 0.1 LU. Going to 120,000 examples gives no further gain, so at this training budget the network, not the data, is the limit. Compared with the baseline on the same new targets, it is 24× more accurate at the median (0.76 to 0.032 LU) and 177× more accurate at the 95th percentile (11.9 to 0.068 LU). It also uses 42% fewer parameters.

C. The network as a starting guess

A network can also give the numerical solver its starting point, while the solver still guarantees the final accuracy. Table III shows the effect. Starting from all-zero joints, the solver needs 15.8 iterations on average, and only 39.5% of its answers are of the intended type. The baseline network cuts the iterations to 6.2 but does not change which answers come out, since it learned the solver's mixed choices. The improved network cuts the iterations to 4.5 and makes *every* answer the intended type.

TABLE III

THE LM SOLVER ON 3,000 NEW REACHABLE TARGETS WITH DIFFERENT STARTING GUESSES. "INTENDED" IS THE SHARE OF ANSWERS WITH THE SHOULDER FACING THE TARGET AND THE ELBOW UP. NO RUN FAILED.

Starting guess	Iterations		Intended
	mean	p95	[%]
All zeros (baseline)	15.8	28	39.5
Baseline network	6.2	11	39.7
Improved network	4.5	5	100.0

Speed. In Python on an Apple M1 Pro CPU, the exact formula takes 12.7 µs per target, the improved network 17.8 µs, the baseline network 50.9 µs, and the LM solver 358 µs from zero or 89 µs from the network's guess. For an arm with an exact formula, the formula is fastest and exact. For arms without one, a network is a cheap way to speed up and steer the solver.

VI. Discussion

Lessons. Each problem we found comes from a default that looks harmless on its own. We suggest the following checklist for anyone training an IK network from solver output:

1. Sample targets the arm can reach, or sample joints and use FK, which gives exact labels for free.
2. Give every target the same type of solution, or use a model built for multiple answers [20], [22].
3. Wrap angles and predict their sine and cosine.
4. Measure where the hand lands, and report both typical and worst-case errors. Do not use a classification "accuracy" for this regression task.
5. Check that input ranges match their units and cover the whole workspace.
6. Keep test data separate, including for example plots, and compare speed against the best available method.

Limitations. MATLAB was not available for this analysis, so solver statistics come from our Python re-implementation. It matches the original data in failure rate and solution-type mix, but leaves angles unwrapped more often than the toolbox did. All results on the baseline models use their original weights and data. The study covers two 3-joint arms and position-only IK; larger arms with full position and orientation targets are the natural next step. Timings were measured in Python and are only indicative.

VII. Conclusion

Training an IK network on a numerical solver's answers seems simple, but the details of the training data decide how well it works. Our baseline network reached targets with a median error of about 4% of the arm's reach, with much larger worst-case errors, while Keras's default accuracy metric showed none of this. The cause was mainly inconsistent labels: the solver picked different solution types for neighbouring targets and left angles unwrapped. Using one consistent solution type, exact labels, and sine–cosine outputs made the same network $43\times$ more accurate. As a starting guess for a numerical solver, the improved network also cut the iteration count by $3.5\times$ and fixed which solution comes out.

Acknowledgment

This work was carried out for MAE 547 (Modeling and Control of Robots) at Arizona State University. The author thanks Rumitkumar Patel and Priyank Patel for their work on the original project. The analysis was carried out with the help of an AI coding assistant. All results were produced by the released scripts, and the author takes full responsibility for the content.

References

[1] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. Springer, 2009.

[2] D. L. Pieper, "The kinematics of manipulators under computer control," Ph.D. dissertation, Stanford University, 1968.

[3] R. Diankov, "Automated construction of robotic manipulation programs," Ph.D. dissertation, Carnegie Mellon University, 2010.

[4] C. W. Wampler, "Manipulator inverse kinematic solutions based on vector formulations and damped least-squares methods," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 16, no. 1, pp. 93–101, 1986.

[5] Y. Nakamura and H. Hanafusa, "Inverse kinematic solutions with singularity robustness for robot manipulator control," *Journal of Dynamic Systems, Measurement, and Control*, vol. 108, no. 3, pp. 163–171, 1986.

[6] S. R. Buss, "Introduction to inverse kinematics with Jacobian transpose, pseudoinverse and damped least squares methods," University of California, San Diego, Tech. Rep., 2004.

[7] T. Sugihara, "Solvability-unconcerned inverse kinematics by the Levenberg–Marquardt method," *IEEE Transactions on Robotics*, vol. 27, no. 5, pp. 984–991, 2011.

[8] P. Beeson and B. Ames, "TRAC-IK: An open-source library for improved solving of generic inverse kinematics," in *IEEE-RAS International Conference on Humanoid Robots (Humanoids)*, 2015.

[9] A. Guez and Z. Ahmad, "Solution to the inverse kinematics problem in robotics by neural networks," in *IEEE International Conference on Neural Networks*, 1988, pp. 617–624.

[10] A. D'Souza, S. Vijayakumar, and S. Schaal, "Learning inverse kinematics," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2001, pp. 298–303.

[11] A. Csiszar, J. Eilers, and A. Verl, "On solving the inverse kinematics problem using neural networks," in *International Conference on Mechatronics and Machine Vision in Practice (M2VIP)*, 2017, pp. 1–6.

[12] M. I. Jordan and D. E. Rumelhart, "Forward models: Supervised learning with a distal teacher," *Cognitive Science*, vol. 16, no. 3, pp. 307–354, 1992.

[13] P. I. Corke, "A robotics toolbox for MATLAB," *IEEE Robotics & Automation Magazine*, vol. 3, no. 1, pp. 24–32, 1996.

[14] P. Corke, *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*, 2nd ed., ser. Springer Tracts in Advanced Robotics. Springer, 2017, vol. 118.

[15] F. Chollet *et al.*, "Keras," https://keras.io, 2015.

[16] K. Levenberg, "A method for the solution of certain non-linear problems in least squares," *Quarterly of Applied Mathematics*, vol. 2, no. 2, pp. 164–168, 1944.

[17] D. W. Marquardt, "An algorithm for least-squares estimation of nonlinear parameters," *Journal of the Society for Industrial and Applied Mathematics*, vol. 11, no. 2, pp. 431–441, 1963.

[18] E. Oyama, A. Agah, K. F. MacDorman, T. Maeda, and S. Tachi, "A modular neural network architecture for inverse kinematics model learning," *Neurocomputing*, vol. 38–40, pp. 797–805, 2001.

[19] B. Bócsi, D. Nguyen-Tuong, L. Csató, B. Schölkopf, and J. Peters, "Learning inverse kinematics with structured prediction," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2011, pp. 698–703.

[20] C. M. Bishop, "Mixture density networks," Aston University, Tech. Rep. NCRG/94/004, 1994.

[21] L. Ardizzone, J. Kruse, S. Wirkert, D. Rahner, E. W. Pellegrini, R. S. Klessen, L. Maier-Hein, C. Rother, and U. Köthe, "Analyzing inverse problems with invertible neural networks," in *International Conference on Learning Representations (ICLR)*, 2019.

[22] B. Ames, J. Morgan, and G. Konidaris, "IKFlow: Generating diverse inverse kinematics solutions," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7177–7184, 2022.

[23] Y. Zhou, C. Barnes, J. Lu, J. Yang, and H. Li, "On the continuity of rotation representations in neural networks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 5745–5753.

[24] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.

[25] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.